Gestión de pruebas

Nombre del proyecto	ADOPTAFACIL
Nombre del sistema	Sistema de información para la adopción de animales
Versión	1.0
Equipo	Nicolas Alberto Valenzuela Pacheco Brayan Esteban Sanchez Avila Andres Felipe Gamez Arias Midred Callejas Chaparro
Instructor	Ruben Sanabria

1. Introducción
PARTE I. Fundamentos teóricos de la gestión de pruebas
2. Definición de gestión de pruebas
3. Importancia de la gestión de pruebas en proyectos de software
4. Ciclo de vida de las pruebas
    4.1. Planificación de pruebas
    4.2. Diseño de pruebas
    4.3. Implementación de pruebas
    4.4. Ejecución de pruebas
    4.5. Análisis de resultados
    4.6. Mejora continua
PARTE II. Gestión de pruebas en el proyecto AdoptaFácil
5. Organización de pruebas en el proyecto
    5.1. Pruebas unitarias
    5.2. Pruebas de integración
    5.3. Pruebas de rendimiento
6. Herramientas utilizadas
7. Ejecución de pruebas
    7.1. Ejecución de pruebas unitarias e integración
    7.2. Ejecución de pruebas con Pest

7.3. Ejecución de pruebas de rendimiento
8. Control de resultados
9. Reportes de pruebas
10. Seguimiento de errores
11. Mejora continua
12. Conclusiones

---

1. Introducción

La calidad del software es un factor fundamental para garantizar que un sistema funcione correctamente, sea confiable y satisfaga las necesidades de las personas usuarias. Para lograrlo, es necesario implementar procesos estructurados de **gestión de pruebas**, los cuales permiten planificar, ejecutar, monitorear y mejorar las actividades relacionadas con el aseguramiento de la calidad del software.

La **gestión de pruebas** consiste en organizar de manera sistemática todas las actividades relacionadas con las pruebas del sistema, incluyendo la planificación, ejecución, control de resultados, reporte de errores y mejora continua del proceso de testing.

El presente documento describe el proceso de **gestión de pruebas aplicado al sistema AdoptaFácil**, una aplicación web desarrollada con el framework **Laravel**. Durante el desarrollo del sistema se implementaron diferentes tipos de pruebas utilizando herramientas modernas como:

- **PestPHP** para pruebas unitarias.
- **Laravel Testing** para pruebas de integración.
- **k6** para pruebas de rendimiento (carga y estrés).
- **Reportes de pruebas en formato JUnit** para análisis y seguimiento de resultados.

PARTE I. Fundamentos teóricos de la gestión de pruebas

2. Definición de gestión de pruebas

La **gestión de pruebas** es el proceso de planificación, organización, supervisión y control de las actividades relacionadas con las pruebas de software durante el

ciclo de vida de un proyecto.

Este proceso incluye actividades como:

- Definición de estrategias de pruebas.
- Organización de los casos de prueba.
- Ejecución de pruebas automatizadas o manuales.
- Análisis de resultados.
- Seguimiento de errores.
- Generación de reportes de calidad.

La gestión de pruebas tiene como objetivo garantizar que el software cumpla con los requisitos funcionales y no funcionales definidos durante el desarrollo.

3. Importancia de la gestión de pruebas en proyectos de software

La gestión de pruebas desempeña un papel clave dentro del desarrollo de software, ya que permite asegurar que el sistema sea confiable, seguro y funcional.

Entre los principales beneficios de implementar una gestión de pruebas adecuada se encuentran:

- Identificación temprana de errores en el sistema.
- Reducción de riesgos en el despliegue del software.
- Mejora de la calidad del producto final.
- Optimización del proceso de desarrollo.
- Documentación clara del estado del sistema.

Además, una correcta gestión de pruebas facilita la comunicación entre las personas miembros del equipo de desarrollo y permite tomar decisiones informadas sobre el estado del proyecto.

4. Ciclo de vida de las pruebas

El **ciclo de vida de las pruebas de software** describe las diferentes etapas que se deben seguir durante el proceso de testing.

4.1. Planificación de pruebas

En esta fase se definen:

- Estrategia de pruebas.
- Alcance de las pruebas.
- Herramientas a utilizar.
- Recursos necesarios.

4.2. Diseño de pruebas

Durante esta fase se diseñan los casos de prueba y se identifican los escenarios que deben ser evaluados.

4.3. Implementación de pruebas

En esta etapa se desarrollan los scripts o casos de prueba que se utilizarán para validar el sistema.

4.4. Ejecución de pruebas

Se ejecutan los casos de prueba definidos y se registran los resultados obtenidos.

4.5. Análisis de resultados

Se analizan los resultados de las pruebas para identificar posibles errores o comportamientos inesperados del sistema.

4.6. Mejora continua

A partir de los resultados obtenidos se realizan mejoras en el sistema y en el proceso de pruebas.

PARTE II. Gestión de pruebas en el proyecto AdoptaFácil

5. Organización de pruebas en el proyecto

Las pruebas del sistema AdoptaFácil están organizadas dentro de una estructura de directorios que permite clasificar los diferentes tipos de pruebas.

Estructura general del proyecto:

```
tests
├── Unit
├── Feature
├── Performance
```

5.1. Pruebas unitarias

Ubicación: `tests/Unit`

Herramienta utilizada:

- PestPHP

Estas pruebas validan componentes individuales del sistema.

5.2. Pruebas de integración

Ubicación: `tests/Feature`

Herramientas utilizadas:

- Laravel Testing
- PestPHP

Estas pruebas verifican la interacción entre diferentes módulos del sistema.

5.3. Pruebas de rendimiento

Ubicación: `tests/Performance`

Herramienta utilizada:

- k6

Estas pruebas evalúan el comportamiento del sistema bajo diferentes niveles de carga.

6. Herramientas utilizadas

Durante el proceso de gestión de pruebas del proyecto se utilizaron diversas herramientas especializadas.

Herramienta	Uso
PestPHP	Pruebas unitarias
Laravel Testing	Pruebas de integración
k6	Pruebas de carga y estrés
JUnit Reports	Generación de reportes de pruebas

Estas herramientas permiten automatizar el proceso de pruebas y analizar los resultados obtenidos.

7. Ejecución de pruebas

Las pruebas del sistema pueden ejecutarse mediante comandos del entorno de desarrollo.

7.1. Ejecución de pruebas unitarias e integración

```
php artisan test
```

Este comando ejecuta todas las pruebas definidas en el proyecto.

7.2. Ejecución de pruebas con Pest

```
./vendor/bin/pest
```

Este comando ejecuta las pruebas escritas utilizando el framework PestPHP.

7.3. Ejecución de pruebas de rendimiento

Las pruebas de rendimiento se ejecutan utilizando k6.

Ejemplo:

```
k6 run load-test.js
```

8. Control de resultados

El control de resultados es una etapa fundamental dentro de la gestión de pruebas.

Durante la ejecución de las pruebas se registran diferentes resultados como:

- Número de pruebas ejecutadas.
- Pruebas exitosas.
- Pruebas fallidas.
- Tiempo de ejecución.

Estos resultados permiten evaluar el estado del sistema y detectar posibles errores.

9. Reportes de pruebas

Para facilitar el análisis de los resultados, el sistema genera reportes en formato **JUnit**.

El formato **JUnit** es ampliamente utilizado en herramientas de integración continua y permite visualizar los resultados de las pruebas de forma estructurada.

Un reporte típico incluye información como:

- Número total de pruebas ejecutadas.
- Número de pruebas exitosas.
- Número de pruebas fallidas.
- Tiempo total de ejecución.

Ejemplo de estructura de reporte:

```
<testsuite name="AdoptaFacilTests" tests="20" failures="0" ti
me="1.23">
```

```
    <testcase classname="AuthTest" name="user_can_login"/>
</testsuite>
```

Este formato permite integrar fácilmente los resultados con herramientas de monitoreo y análisis.

10. Seguimiento de errores

Durante la ejecución de las pruebas se pueden identificar diferentes tipos de errores en el sistema. Los errores detectados deben registrarse y documentarse para su posterior corrección.

El proceso de seguimiento de errores incluye:

1. Identificación del error.
2. Registro del error.
3. Análisis de la causa.
4. Corrección del problema.
5. Validación mediante nuevas pruebas.

Este proceso permite garantizar que los errores detectados sean corregidos antes de que el sistema sea desplegado en producción.

11. Mejora continua

La mejora continua es un principio fundamental dentro de la gestión de pruebas.

A partir de los resultados obtenidos durante las pruebas es posible:

- Optimizar el rendimiento del sistema.
- Mejorar la cobertura de pruebas.
- Detectar debilidades en la arquitectura del sistema.
- Fortalecer la seguridad de la aplicación.

La implementación de procesos de mejora continua permite aumentar progresivamente la calidad del software.

12. Conclusiones

La gestión de pruebas implementada en el proyecto AdoptaFácil permitió organizar de manera estructurada todas las actividades relacionadas con el aseguramiento de la calidad del sistema.

El uso de herramientas modernas como **PestPHP, Laravel Testing y k6** facilitó la implementación de pruebas automatizadas y permitió evaluar diferentes aspectos del sistema, incluyendo funcionalidad, integración y rendimiento.

La generación de reportes en formato **JUnit** permitió analizar los resultados de las pruebas de manera clara y estructurada, facilitando el seguimiento del estado del sistema.

En conclusión, la implementación de una gestión de pruebas adecuada contribuye significativamente a mejorar la calidad del software y a reducir riesgos durante el proceso de desarrollo y despliegue del sistema.